

4. Math and Number Theory

Contents

Math	1
ModInt	5
Sieve / PHI up to n / 2D gcd()	6
Sieve up to 1e9	8
Egcd, Linear Diophantine	9
CRT	10
MillerRabin	10
Number of Divisors up to 1e18	11
$n \bmod 1 + n \bmod 2 + n \bmod 3 + \dots + n \bmod m$	13
n-th Fib Number	13
Long Division	13
Floor Values	13
Combinatorics	13
nCr , nPr without precomputation	14
NCR table	14
Catalan numbers	15
Matrix Exponentiation	16
K-th permutation	17
Permutation Index	17
Berlekamp Massey	17
Product of divisors	19

Math

- 1 Stirling numbers of the first kind : the number of permutations
- 2 of n elements with k disjoint cycles
- 3 $S(n,k) = n * S(n-1,k) + S(n-1,k-1)$
- 4 Sum $k = 0 \rightarrow n$ of $S(n,k) = n!$

5

6 Stirling numbers of the second kind : the number of ways to partition
7 a set of n elements into k nonempty subsets
8 $S(n,k) = k * S(n-1,k) + S(n-1,k-1)$
9 $\sum_k = 0 \rightarrow n$ of $S(n,k) = B_n$

10

11 Bell numbers : the possible partitions of a set into nonempty subsets
12 $\sum_k = 0 \rightarrow n-1$ of $n-1Ck * B_k = B_n$

13

14 Sum of first n even numbers : $n*(n+1)$
15 Sum of first n odd numbers : $n*n$

16

17 Sum of squares of first n numbers:
18 $n*(n+1)*(2*n+1)/6$
19 Sum of squares of first n even numbers:
20 $2*n*(n+1)*(2*n+1)/3$
21 Sum of squares of first n odd numbers:
22 $n*(2*n+1)*(2*n-1)/3$

23

24 Number of ways to pick equal number of elements from two sets : $(n+m)C(m)$

25

26 Sum of $\phi(d)$ for all $d | n$ is equal to n .
27 Number of pairs (x,y) that satisfy $x+y=n$ and $\gcd(x,y)=1$ is $\phi(n)$.

28

29 $\sum (nCk)$ for $k [0,n] = 2^n$
30 $\sum (mCk)$ for all $m [0,n] = (n+1)C(k+1)$
31 $\sum ((n+k)Ck)$ for all $k [0,m] = (n+m+1)C(m)$
32 $\sum ((nCk)^2)$ for all $k [0,n] = (2*n)Cn$
33 $\sum (i*nCi)$ for all $i [1,n] = n*2^{(n-1)}$
34 $\sum ((n-i)Ci)$ for all $i [0,n] = F(n+1)$
35 Number of arrays with size n and sum $m = (n-1+m)C(m) = (n-1+m)C(n-1)$

36

37 $P(A|B)$ is the probability of event A given that event B happened.
38 $P(A \& B)$ is the probability of events A and B happening.

39

40 $P(A|B) = (P(B|A) * P(A)) / P(B)$
41 $P(A|B) = P(A \& B) / P(B)$

42

43 Divisibility:
44 2 if the rightmost digit is divisible by 2
45 3 if the sum of the digits is divisible by 3
46 4 if the number formed by the last two digits is divisible by 4
47 5 if the rightmost digit is 0 or 5
48 6 if it is divisible by 2 and 3
49 7 if The number formed by all digits except the right-most digit -
50 $(2 * \text{right-most digit})$ is divisible by 7
51 8 if the number formed by the last 3 digits is divisible by 8
52 9 if the sum of the digits is divisible by 9

53

54 -Getting half of the binomial expansion (Only odd indices or only even indices)
55 by using $(a+b)^n$ and $(a-b)^n$ and adding both of them

56

57 To solve quadratic equation $a*x^2 + b*x + c = 0$
58 $x = \frac{-b (+/-)\sqrt{b^2-4*a*c}}{(2 * a)}$

59

60 -Sum of geometric series ar^i (i from 0 to n) = $a(1 - r^n) / (1 - r)$
61 -Sum of geometric series ar^i (i from 0 to infinity) = $a / (1 - r)$

```

62
63 - XOR from 1 to n = n%4 [n, 1, n+1, 0]
64
65 - n can be sum of 2 squares if n has no p^k [p = 3 % 4, k is odd]
66   in its prime factorization
67 - n can be sum of 3 squares if n is not in form [4^a (8b + 7)]
68 - n always can be sum of >= 4 squares (remaining are zeros)
69
70 ===== FADY THE GO0000AAT =====
71 - sum of divisors
72   prime^power * prime2^power2 * ...
73
74   ((prime^(power + 1) - 1) / (prime - 1)) * ((prime2^(power2 + 1) - 1) / (prime2 - 1)) * ...
75 =====
76   a % m == b
77   a and m not coprime
78   g = gcd(a, m)
79   (a / g) % (m / g) = b / g
80
81   a^x % m == b
82   a and m not coprime
83   g = gcd(a, m)
84   (a^(x-1) * (a / g)) % (m / g) = b / g
85 =====
86   a^(power%phi(m)) % m;
87 =====
88   count balanced brackets
89   r=n/2 || or r = number of opened brackets
90   nCr(n, r) - nCr(n, r-1)
91 =====
92   // different n*n grids whose each square have m colors
93   // if possible to rotate one of them so that they look the same then they same
94   t = n * n;
95   total = (fp(m, t)
96           + fp(m, (t + 1) / 2)
97           + 2 * fp(m, (t / 4) + (n % 2))) % mod;
98   total = mul(total, fp(4, mod - 2));
99 =====
100  biggest divisors
101  735134400 1344 => 2^6 3^3 5^2 7 11 13 17
102  73513440 768
103  =====
104  for (int x = mask; x > 0; x = (x - 1) & mask)
105  get all x such that mask = mask | x
106  =====
107  sum from 1 to n: i * nCr(n, i) = n * (1LL << (n - 1))
108  sum of odd between [1, n] = ((n+1) / 2)^2
109  sum of even between [1, n] = (n/2) * (n/2 + 1)
110
111 ll sum_total(ll l, ll r) { // sum [l, r]
112     return (r - l + 1) * (l + r) / 2;
113 }
114
115 ll sum_even(ll l, ll r) { // sum even [l, r]
116     if (l % 2 != 0) ++l;
117     if (r % 2 != 0) --r;
118     if (l > r) return 0;

```

```

119     ll n = (r - l) / 2 + 1;
120     return n * (l + r) / 2;
121 }
122
123 ll sum_odd(ll l, ll r) { // sum odd [l, r]
124     if (l % 2 == 0) ++l;
125     if (r % 2 == 0) --r;
126     if (l > r) return 0;
127     ll n = (r - l) / 2 + 1;
128     return n * (l + r) / 2;
129 }
130
131 ll arithm1(ll l, ll r, ll a, ll d) { // [l, r] starting a and diff d
132     if (d == 0) return (a >= l && a <= r) ? a : 0;
133     ll n1 = ((l - a + d - (d > 0 ? 1 : -1)) / d);
134     ll first = a + n1 * d;
135     if ((d > 0 && first > r)
136         || (d < 0 && first < r))
137         return 0;
138     ll n2 = ((r - a) / d);
139     ll last = a + n2 * d;
140     ll n = (n2 - n1 + 1);
141     return n * (first + last) / 2;
142 }
143
144 ll arithm2(ll a, ll d, ll n) { // starting a, diff d, n terms
145     return n * (2 * a + (n - 1) * d) / 2;
146 }
147 */
148
149 ll phi(ll x) { // sqrt(x)
150     ll ans = x;
151     for(ll i = 2; i * i <= x; i++) {
152         if(x % i == 0) {
153             while(x % i == 0) x /= i;
154             ans -= ans / i;
155         }
156     }
157     if(x > 1) ans -= ans / x;
158     return ans;
159 }
160
161 array<ll, 2> CRT(ll a1, ll m1, ll a2, ll m2) {
162     // x = a1 % m1, x = a2 % m2
163     a1 %= m1, a2 %= m2;
164     auto [g, q1, q2] = eGcd(m1, -m2);
165     if ((a2 - a1) % g) return {-1, -1};
166     ll lcm = m1 / g * m2;
167     ll m = m2 / g;
168     q1 = (a2 - a1) / g % m * q1 % m;
169     ll res = (a1 + m1 * q1) % lcm;
170     if (res < 0) res += lcm;
171     return {res, lcm};
172 }
173
174 ll BSGS(ll a, ll b, ll p) { // a^x = b (mod p)

```

```

175     a %= p, b %= p;
176     if(b == 1) return 0;
177     if(a == 0) return b == 0? 1: -1;
178     int add = 0;
179     ll g, tmp = 1;
180     while ((g = gcd(a, p)) > 1) {
181         if(b % g) return -1;
182         p /= g, b /= g, tmp = tmp * (a / g) % p, ++add;
183         if(tmp == b) return add;
184     }
185     b = b * modInv(tmp, p) % p;
186     int n = (int)sqrtl(p) + 1;
187     unordered_map<ll, int> mp;
188     for (ll q = 0, cur = 1; q <= n; ++q)
189         mp.emplace(cur, q), cur = cur * a % p;
190     ll an = 1;
191     for (ll i = 0; i < n; ++i) an = an * a % p;
192     an = modInv(an, p);
193     for (ll i = 0, cur = b; i <= n; ++i) {
194         auto it = mp.find(cur);
195         if(it != mp.end()) return i * n + it->second + add;
196         cur = cur * an % p;
197     }
198     return -1;
199 }
200
201 int sumNPowerM(int n, int m) { // 1^m + 2^m ... n^m
202     int k = m + 3;
203     vector<int> res(k);
204     for(int i = 1; i < k; i++) res[i] = (res[i - 1] + fp(i, m)) % mod;
205     if(n < k) return res[n];
206     int facK = k;
207     vector<int> p(k); p[0] = 1;
208     for(int i = 1; i < k; i++) {
209         p[i] = int(p[i - 1] * 1LL * (n - i) % mod);
210         facK = int(facK * 1LL * i % mod);
211     }
212     vector<int> inv(k + 1), s(k + 1);
213     inv[k] = fp(facK, mod - 2), s[k] = 1;
214     for(int i = k - 1; i >= 0; i--) {
215         s[i] = int(s[i + 1] * 1LL * (n - i) % mod);
216         inv[i] = int(inv[i + 1] * 1LL * (i + 1) % mod);
217     }
218     int ans = 0;
219     for(int i = 1; i < k; i++) {
220         int cur = int(res[i] * 1LL * p[i - 1] % mod * s[i + 1] % mod *
221             inv[i - 1] % mod * inv[k - i - 1] % mod);
222         if((k - i + 1) & 1) cur = (mod - cur) % mod;
223         ans = (ans + cur) % mod;
224     }
225     return ans;
226 }

```

ModInt

```

1  template <int M> struct ModInt {
2      template <class T> T binpow(T a, ll b) {
3          T res = 1;
4          while (b) { if (b & 1) res *= a; a *= a, b >>= 1; }
5          return res;
6      }
7      int v;
8      ModInt() : v(0) {}
9      ModInt(ll v_) {
10         v = v_ % M;
11         if (v < 0) v += M;
12     }
13
14     bool operator==(ModInt o) const { return v == o.v; };
15     bool operator!=(ModInt o) const { return !(*this == o); };
16
17     // ModInt add(ModInt a, ModInt b) { return ((a.v + b.v % M)+M)%M;}
18     // ModInt sub(ModInt a, ModInt b) { return add(a, -b); }
19     // ModInt mul(ModInt a, ModInt b) { return 1ll * a.v * b.v % M; }
20     // ModInt div(ModInt a, ModInt b) { return mul(a, binpow(b, M-2)); }
21
22     ModInt &operator+=(ModInt o) { v = (v + o.v) % M; return *this; }
23     ModInt &operator-=(ModInt o) { v = (v - o.v + M) % M; return *this; }
24     ModInt &operator*=(ModInt o) { v = 1ll * v * o.v % M; return *this; }
25     ModInt &operator/=(ModInt o) { return (*this *= binpow(o, M - 2)); }
26
27     friend ModInt operator+(ModInt a, ModInt b) { return a += b; }
28     friend ModInt operator-(ModInt a, ModInt b) { return a -= b; }
29     friend ModInt operator*(ModInt a, ModInt b) { return a *= b; }
30     friend ModInt operator/(ModInt a, ModInt b) { return a /= b; }
31     ModInt operator-() { return 0 - *this; }
32
33     friend istream &operator>>(istream &is, ModInt &a) {
34         ll x; is >> x;
35         a = ModInt(x);
36         return is;
37     }
38     friend ostream &operator<<(ostream &os, ModInt a) { return os << a.v; }
39     friend string to_string(ModInt a) { return to_string(a.v); }
40 };
41 using mint = ModInt<>;

```

Sieve / PHI up to n / 2D gcd()

```

1  const int NS = 1e7;
2  const int NP = (NS/log(NS)) * (1 + 1.28 / log(NS));
3  int prSz;
4  int spf[NS], prm[NP];
5
6  auto pre_Sieve = []() {
7      for (int i = 2; i < NS; i++){
8          if(!spf[i]) spf[i] = prm[prSz++] = i;
9          for(int j = 0; i * prm[j] < NS; j++) {
10             spf[i * prm[j]] = prm[j];
11             if(spf[i] == prm[j]) break;

```

```

12     }
13 }
14 return 0;
15 }();
16
17 auto factors(int n) {
18     vector<array<int, 2>> res;
19     if(n < 2) return res;
20     int p = spf[n];
21     while(p > 1) {
22         res.push_back({p, 0});
23         while(n % p == 0) n /= p, res.back()[1]++;
24         p = spf[n];
25     }
26     return res;
27 }
28
29 auto getDivisors(int _n) {
30     auto _fac = factors(_n);
31     int cnt = 1;
32     for(auto [pr, pw] : _fac) cnt *= pw + 1;
33     vector<int> res(1, 1); res.reserve(cnt);
34
35     for(auto [pr, pw] : _fac)
36         for(int i = int(res.size()) - 1; i >= 0; i--)
37             for(int b = pr, j = 0; j < pw; j++, b *= pr)
38                 res.push_back(res[i] * b);
39     sort(res.begin(), res.end());
40     return res;
41 }
42
43 bool isPrime(ll n) {
44     if(n < 4) return n > 1;
45     if(n % 2 == 0 || n % 3 == 0) return false;
46     for (ll i = 5; i * i <= n; i += 6)
47         if (n % i == 0 || n % (i + 2) == 0)
48             return false;
49     return true;
50 }
51
52 void phi_1_to_n(int n) {
53     vector<int> phi(n + 1);
54     for (int i = 0; i <= n; i++)
55         phi[i] = i;
56
57     for (int i = 2; i <= n; i++) {
58         if (phi[i] == i) {
59             for (int j = i; j <= n; j += i)
60                 phi[j] -= phi[j] / i;
61         }
62     }
63 }
64
65 // 2d gcd in n^2
66 vector<vector<int>> GCD(N, vector<int>(N, 1));
67 for(int d = 2; d < N; ++d)

```

```

68     for(int i = d; i < N; i += d)
69         for(int j = d; j < N; j += d)
70             GC[i][j] = d;

```

Sieve up to 1e9

```

1  vector<int> sieve(const int BN, const int Q = 17, const int L = 1 << 15) {
2      using u = uint32_t; using u8 = uint8_t;
3      static const u rs[] = {1, 7, 11, 13, 17, 19, 23, 29};
4      struct P {
5          u p;
6          u pos[8];
7      };
8
9      const u v = sqrt(BN), vv = sqrt(v);
10     vector<bool> isp(v + 1, true);
11     for (u i = 2; i <= vv; ++i)
12         if (isp[i])
13             for (u j = i * i; j <= v; j += i) isp[j] = false;
14
15     const u rsize = BN > 60184 ? BN / (log(BN) - 1.1) :
16         max(1.0, BN / (log(BN) - 1.11)) + 1 + 30;
17     vector<int> primes = {2, 3, 5}; u psize = 3;
18     primes.resize(rsize); vector<P> sprimes;
19
20     u pbeg = 0, prod = 1;
21     for (u p = 7; p <= v; ++p) {
22         if (!isp[p]) continue;
23         if (p <= Q) prod *= p, ++pbeg, primes[psize++] = p;
24         P pp = {p, {}};
25         for (int t = 0; t < 8; ++t) {
26             u j = p <= Q ? p : p * p;
27             while (j % 30 != rs[t]) j += p << 1;
28             pp.pos[t] = j / 30;
29         }
30         sprimes.push_back(pp);
31     }
32
33     vector<u8> pre(prod, 0xFF);
34     for (size_t pi = 0; pi < pbeg; ++pi) {
35         auto &pp = sprimes[pi];
36         const u p = pp.p;
37         for (int t = 0; t < 8; ++t) {
38             const u8 m = ~(1 << t);
39             for (u i = pp.pos[t]; i < prod; i += p) pre[i] &= m;
40         }
41     }
42
43     const u block_size = (L + prod - 1) / prod * prod;
44     vector<u8> block(block_size);
45     u8* __restrict pblock = block.data();
46     const u M = (BN + 29) / 30;
47
48     for (u beg = 0; beg < M; beg += block_size, pblock -= block_size) {
49         u end = min(M, beg + block_size);

```



```

50
51     for (u i = beg; i < end; i += prod)
52         memcpy(pblock + i, pre.data(), min(prod, end - i));
53     if (beg == 0) pblock[0] &= 0xFE;
54
55     for (size_t pi = pbeg; pi < sprimes.size(); ++pi) {
56         auto &pp = sprimes[pi];
57         const u p = pp.p;
58         #pragma GCC unroll 8
59         for (int t = 0; t < 8; ++t) {
60             u i = pp.pos[t];
61             const u8 m = ~(1 << t);
62             for (; i < end; i += p) pblock[i] &= m;
63             pp.pos[t] = i;
64         }
65     }
66
67     for (u i = beg; i < end; ++i)
68         for (u m = pblock[i]; m > 0; m &= m - 1)
69             primes[psize++] = i * 30 + rs[__builtin_ctz(m)];
70 }
71
72 while (psize > 0 && primes[psize - 1] > BN) --psize;
73 primes.resize(psize); return primes;
74 }

```

Egcd, Linear Diophantine

```

1  array<ll, 3> eGcd(ll a, ll b) {
2      if (b == 0) return {a, 1, 0};
3      auto [g, x1, y1] = eGcd(b, a % b);
4      return {g, y1, x1 - (a / b) * y1};
5  }
6
7  ax0 + by0 = g ==> ax + by = c
8  x0 *= c/g, y0 *= c/g
9
10 all solutions:
11     x = x0 + k * (b/g),
12     y = y0 - k * (a/g);
13
14 int ceildiv(int a, int b) {
15     if ((a ^ b) >= 0) return (a + b - 1) / b;
16     else return a / b;
17 }
18
19 int flordiv(int a, int b) {
20     if ((a ^ b) >= 0) return a / b;
21     else return (a - b + 1) / b;
22 }
23
24 non-negative solution:
25     k >= ceildiv(-x*g, b)
26     k <= flordiv(x*g, a)
27

```

```

28 bool havenonnegsol(int a, int b, int c, int& x, int& y) {
29     int g = egcd(a, b, x, y);
30     x *= c/g;
31     y *= c/g;
32     int l1 = ceildiv(-x*g, b);
33     int l2 = flordiv(y*g, a);
34     return l1 <= l2;
35 }
36
37 // MOD INV
38 ll modInv(ll a, ll m) {
39     ll x = 1, x1 = 0, q, t, b = m;
40     while(b) {
41         q = a / b;
42         a -= q * b, t = a, a = b, b = t;
43         x -= q * x1, t = x, x = x1, x1 = t;
44     }
45     assert(a == 1);
46     return (x + m) % m;
47 }

```

CRT

```

1  using T = __int128;
2  // ax + by = __gcd(a, b)
3  // returns __gcd(a, b)
4  T extended_euclid(T a, T b, T &x, T &y) {
5      T xx = y = 0;
6      T yy = x = 1;
7      while (b) {
8          T q = a / b;
9          T t = b; b = a % b; a = t;
10         t = xx; xx = x - q * xx; x = t;
11         t = yy; yy = y - q * yy; y = t;
12     }
13     return a;
14 }
15 // finds x such that x % m1 = a1, x % m2 = a2. m1 and m2 may not be coprime
16 // here, x is unique modulo m = lcm(m1, m2). returns (x, m). on failure, m = -1.
17 pair<T, T> CRT(T a1, T m1, T a2, T m2) {
18     T p, q;
19     T g = extended_euclid(m1, m2, p, q);
20     if (a1 % g != a2 % g) return make_pair(0, -1);
21     T m = m1 / g * m2;
22     p = (p % m + m) % m;
23     q = (q % m + m) % m;
24     return make_pair((p * a2 % m * (m1 / g) % m +
25                     q * a1 % m * (m2 / g) % m) % m, m);
26 }

```

MillerRabin

```

1  ll binpower(ll base, ll e, ll mod) {
2      ll result = 1;

```

```

3     base %= mod;
4     while (e) {
5         if (e & 1)
6             result = (__uint128_t)result * base % mod;
7         base = (__uint128_t)base * base % mod;
8         e >>= 1;
9     }
10    return result;
11 }
12
13 bool check_composite(ll n, ll a, ll d, int s) {
14     ll x = binpower(a, d, n);
15     if (x == 1 || x == n - 1)
16         return false;
17     for (int r = 1; r < s; r++) {
18         x = (__uint128_t)x * x % n;
19         if (x == n - 1)
20             return false;
21     }
22     return true;
23 };
24
25 bool MillerRabin(long long n) {
26     if (n < 2)
27         return false;
28
29     vector<int> primes = {2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31, 37};
30     for (auto a : primes)
31         if (n % a == 0)
32             return false;
33
34     int r = 0;
35     long long d = n - 1;
36     while ((d & 1) == 0) {
37         d >>= 1;
38         r++;
39     }
40
41     for (int a : primes) {
42         if (n == a)
43             return true;
44         if (check_composite(n, a, d, r))
45             return false;
46     }
47     return true;
48 }

```

Number of Divisors up to 1e18

```

1 namespace countdivisors {
2     // for one second: 5000 1e18, 100000 1e9, 200000 1e6
3     using ull = unsigned long long;
4     using u128 = __uint128_t;
5     static mt19937_64 rnd(chrono::steady_clock::now().time_since_epoch().count());
6     u128 mul128(ull a, ull b, ull m) { return (u128)a * b % m; }
7     ull mul_mod(ull a, ull b, ull m) { return (ull)mul128(a, b, m); }

```

```

8   ull pow_mod(ull a, ull d, ull m) {
9       ull r = 1;
10      while (d) {
11          if (d & 1) r = mul_mod(r, a, m);
12          a = mul_mod(a, a, m);
13          d >>= 1;
14      }
15      return r;
16  }
17  bool isPrime(ull n) {
18      if (n < 2) return false;
19      for (ull p : vector<ull>({2ULL,3,5,7,11,13,17,19,23,29,31,37}))
20          if (n % p == 0) return n == p;
21      ull d = n - 1, s = 0;
22      while (!(d & 1)) d >>= 1, ++s;
23      for (ull a : vector<ull>({2ULL,325,9375,28178,450775,9780504,1795265022})) {
24          if (a % n == 0) continue;
25          ull x = pow_mod(a, d, n);
26          if (x == 1 || x == n - 1) continue;
27          bool comp = true;
28          for (ull r = 1; r < s; ++r) {
29              x = mul_mod(x, x, n);
30              if (x == n - 1) { comp = false; break; }
31          }
32          if (comp) return false;
33      }
34      return true;
35  }
36  ull pollards_rho(ull n) {
37      if (n % 2 == 0) return 2;
38      while (true) {
39          ull c = uniform_int_distribution<ull>(1, n - 1)(rnd);
40          auto f = [&](ull x){ return (mul_mod(x, x, n) + c) % n; };
41          ull x = rnd() % n, y = x, d = 1;
42          while (d == 1) {
43              x = f(x); y = f(f(y));
44              d = gcd<ull>(x > y ? x - y : y - x, n);
45          }
46          if (d < n) return d;
47      }
48  }
49  void factor(ull n, map<ull,int>& cnt) {
50      if (n < 2) return;
51      if (isPrime(n)) { cnt[n]++; return; }
52      ull d = pollards_rho(n);
53      factor(d, cnt);
54      factor(n/d, cnt);
55  }
56  ull ans(ull n) {
57      map<ull,int> cnt;
58      factor(n, cnt);
59      ull res = 1;
60      for (auto [p, e] : cnt) res *= (e + 1);
61      return res;
62  }
63  }

```

$n \bmod 1 + n \bmod 2 + n \bmod 3 + \dots + n \bmod m$

```

1 // n and m tends to(1e13) => time complexity(sqrt(n))
2 void solve(int idx){
3     int k=n/idx;
4     int st=n%idx%mod;
5     int l=n/(k+1);
6     int len=(idx-l)%mod;
7     ans=(ans + st*len%mod + k%mod*(len*(len-1)/2%mod)%mod)%mod;
8     if(l) solve(l);
9 }

```

n-th Fib Number

```

1 int fast_Fibonacci(int n) {
2     int i = 1, h = 1, j = 0, k = 0, t;
3     while (n > 0) {
4         if (n % 2 == 1)
5             t = j * h, j = i * h + j * k + t, i = i * k + t;
6             t = h * h, h = 2 * k * h + t, k = k * k + t, n = n / 2;
7     }
8     return j;
9 }

```

Long Division

```

1 int a = 23, b = 5, n = 10;
2 vector<int> s;
3 for (int i = 0; i < n; i++) {
4     unsigned long long k = a / b; // Integer division: gets the next digit
5     a -= b * k;                    // Remove the part already represented
6     a *= 10;                       // Shift remainder left (for next digit)
7     s.push_back(k);                // Store the digit
8 }

```

Floor Values

```

1 // code to get all different values of floor(n/i)
2 for (ll l = 1, r = 1; (n/l); l = r + 1) { // O(sqrt)
3     r = (n/(n/l));
4     // q = (n/l), process the range [l, r]
5 }

```

Combinatorics

```

1 namespace combinatorics {
2     vector<mint> fac_(1, 1), inv_(1, 1);
3     void build(int N) {
4         N += 10;
5         fac_.assign(N, 1);
6         inv_.assign(N, 1);

```

```

7         for (int i = 2; i < N; i++)
8             fac_[i] = fac_[i-1] * i;
9         inv_[N-1] = 1/fac_[N-1];
10        for (int i = N-2; i > 1; i--)
11            inv_[i] = inv_[i+1] * (i+1);
12    }
13    inline mint nCr(int n, int r) {
14        if(n < 0 || r < 0 || r > n) return 0;
15        return fac_[n] * inv_[r] * inv_[n - r];
16    }
17
18    inline mint nCr(int64_t n, int64_t r) {
19        if(n < 0 || r < 0 || r > n) return 0;
20        r = min<int64_t>(r, n - r);
21        mint ans = inv_[r];
22        for(int64_t i = n - r + 1; i <= n; i++) ans = ans * i;
23        return ans;
24    }
25    inline mint nPr(int n, int r) {
26        if(n < 0 || r < 0 || r > n) return 0;
27        return fac_[n] * inv_[n - r];
28    }
29    inline mint stars_andBars(int n, int r){
30        return nCr(n + r - 1, r - 1);
31    }
32    inline mint catalan(int n) {
33        if(n < 0) return 0;
34        return fac_[2 * n] * inv_[n] * inv_[n + 1];
35    }
36 }
37 // using namespace combinatorics;
38 auto pre = []() { combinatorics::build(); return 0; }();

```

nCr, nPr without precomputation

```

1  ll nCr(ll n, ll r) {
2      if (r < 0 || r > n) return 0;
3      r = min(r, n-r);
4      ll ret = 1;
5      for (int i = 0; i < r; i++)
6          ret = ret * (n-i) / (i+1);
7      return ret;
8  }
9  ll nPr(int n, int r) {
10     if (r < 0 || r > n) return 0;
11     ll ret = 1;
12     for (int i = 0; i < r; ++i) {
13         ret *= (n - i);
14     }
15     return ret;
16 }

```

NCR table

```

1 void preprocess_nCr() {
2     for (int n = 0; n < N; n++) {
3         C[n][0] = C[n][n] = 1;
4         for (int r = 1; r < n; r++) {
5             C[n][r] = (C[n - 1][r - 1] + C[n - 1][r]) % MOD;
6         }
7     }
8 }

```

Catalan numbers

```

1 const int MOD = ....
2 const int MAX = ....
3 int catalan[MAX];
4 void init() {
5     catalan[0] = catalan[1] = 1;
6     for (int i=2; i<=n; i++) {
7         catalan[i] = 0;
8         for (int j=0; j < i; j++) {
9             catalan[i] += (catalan[j] * catalan[i-j-1])% MOD;
10            if (catalan[i] >= MOD)
11                catalan[i] -= MOD;
12        }
13    }
14 }
15
16 // 1- Number of correct bracket sequence consisting of n opening and n
17 // closing brackets.
18
19 // 2- The number of rooted full binary trees with n+1 leaves (vertices
20 // are not numbered).
21 // A rooted binary tree is full if every vertex has
22 // either two children or no children.
23
24 // 3- The number of ways to completely parenthesize n+1 factors.
25
26 // 4- The number of triangulations of a convex polygon with n+2 sides
27 // (i.e. the number of partitions of polygon into disjoint triangles by
28 // using the diagonals).
29
30 // 5- The number of ways to connect the 2n points on a circle to form n
31 // disjoint chords.
32
33 // 6- The number of non-isomorphic full binary trees with n internal
34 // nodes (i.e. nodes having at least one son).
35
36 // 7- The number of monotonic lattice paths from point (0,0) to point
37 // (n,n) in a square lattice of size nxn,
38 // which do not pass above the main diagonal (i.e. connecting (0,0) to
39 // (n,n)).
40
41 // 8- Number of permutations of length n that can be stack sorted
42 // (i.e. it can be shown that the rearrangement is stack sorted if and
43 // only if there is no such index i<j<k, such that a_k<a_i<a_j).
44
45 // 9- The number of non-crossing partitions of a set of n elements.

```

```

46
47 // 10- The number of ways to cover the ladder 1..n using n rectangles

```

Matrix Exponentiation

```

1  template <typename T>
2  struct Matrix {
3      int n, m;
4      vector<T> mat;
5
6      Matrix(int n, int m) : n(n), m(m) {
7          mat.assign(n * m, 0);
8      }
9      Matrix(int n) : n(n), m(n) {
10         mat.assign(n * n, 0);
11     }
12
13     void identity() {
14         assert(n == m);
15         fill(mat.begin(), mat.end(), 0);
16         for (int i = 0; i < n; i++) {
17             mat[i * m + i] = 1;
18         }
19     }
20
21     T* operator[](int r) { return &mat[r * m]; }
22     const T* operator[](int r) const { return &mat[r * m]; }
23
24     Matrix operator*(const Matrix& o) const {
25         assert(m == o.n);
26         Matrix res(n, o.m);
27
28         for (int i = 0; i < n; i++) {
29             for (int k = 0; k < m; k++) {
30                 T val = mat[i * m + k]; if (val == 0) continue;
31                 for (int j = 0; j < o.m; j++) {
32                     res[i][j] = (res[i][j] + 1ll * val * o[k][j]) % mod;
33                     // res[i][j] = res[i][j] + val * o[k][j];
34                 }
35             }
36         }
37         return res;
38     }
39
40     Matrix operator+(const Matrix& o) const {
41         assert(o.n == n && o.m == m);
42         Matrix ret(n, m);
43         for (int i = 0; i < n; i++)
44             for (int j = 0; j < m; j++) {
45                 ret[i][j] = ((*this)[i][j] + o[i][j]) % mod;
46                 // ret[i][j] = (*this)[i][j] + o[i][j];
47             }
48         return ret;
49     }
50

```



```

51     Matrix pow(ll k) const {
52         assert(n == m);
53         Matrix res(n), base = *this;
54         res.identity();
55         while (k) {
56             if (k & 1) res = res * base;
57             base = base * base;
58             k >>= 1;
59         }
60         return res;
61     }
62 };

```

K-th permutation

```

1  ll fac[21];
2  void preFac() {
3      fac[0] = 1;
4      for(int i = 1; i <= 20; ++i)
5          fac[i] = fac[i - 1] * i;
6  }
7
8  vector<int> kth_permutation(int n, ll k) { // n^2
9      vector<int> a(n), ans;
10     iota(a.begin(), a.end(), 1);
11     for (int i = n; i >= 1; i--) {
12         ll f = fac[i - 1];
13         ans.push_back(a[k / f]);
14         a.erase(a.begin() + k / f);
15         k %= f;
16     }
17     return ans;
18 }

```

Permutation Index

```

1  ll permutation_index(vector<int>& p) { // n^2
2      int n = int(p.size());
3      vector<int> a(n);
4      iota(a.begin(), a.end(), 1);
5
6      ll k = 0;
7      for (int i = 0; i < n; ++i) {
8          int j = int(find(a.begin(), a.end(), p[i]) - a.begin());
9          k += j * fac[n - 1 - i];
10         a.erase(a.begin() + j);
11     }
12     return k;
13 }

```

Berlekamp Massey

```

1  const ll MOD = 1e9+7;
2  ll add(ll a, ll b) { return (a + b) % MOD; }
3  ll sub(ll a, ll b) { return ((a - b) % MOD + MOD) % MOD; }
4  ll mul(ll a, ll b) { return (a * b) % MOD; }
5  ll power(ll base, ll exp) {
6      ll res = 1;
7      base %= MOD;
8      while (exp > 0) {
9          if (exp % 2 == 1) res = mul(res, base);
10         base = mul(base, base);
11         exp /= 2;
12     }
13     return res;
14 }
15 ll modInverse(ll n) {
16     return power(n, MOD - 2); // Valid only if MOD is prime
17 }
18 // Finds the shortest linear recurrence transition array for a given sequence S
19 vector<ll> BerlekampMassey(const vector<ll>& S) {
20     int n = S.size(), L = 0, m = 0;
21     vector<ll> C(n), B(n), T;
22     C[0] = B[0] = 1;
23     ll b = 1;
24
25     for (int i = 0; i < n; i++) {
26         ++m;
27         ll d = S[i] % MOD;
28         for (int j = 1; j <= L; j++) d = add(d, mul(C[j], S[i - j]));
29         if (d == 0) continue;
30
31         T = C;
32         ll coef = mul(d, modInverse(b));
33         for (int j = m; j < n; j++) C[j] = sub(C[j], mul(coef, B[j - m]));
34
35         if (2 * L > i) continue;
36         L = i + 1 - L;
37         B = T;
38         b = d;
39         m = 0;
40     }
41     C.resize(L + 1);
42     C.erase(C.begin());
43     for (auto &x : C) x = sub(0, x);
44     return C;
45 }
46 // Multiplies two polynomials modulo the characteristic polynomial (tr)
47 vector<ll> combine(int n, const vector<ll>& a, const vector<ll>& b, const vector<ll>& tr) {
48     vector<ll> res(n * 2 + 1, 0);
49     for (int i = 0; i < n + 1; i++) {
50         for (int j = 0; j < n + 1; j++) {
51             res[i + j] = add(res[i + j], mul(a[i], b[j]));
52         }
53     }
54     for (int i = 2 * n; i > n; --i) {
55         for (int j = 0; j < n; j++) {
56             res[i - 1 - j] = add(res[i - 1 - j], mul(res[i], tr[j]));

```

```

57     }
58 }
59 res.resize(n + 1);
60 return res;
61 }
62 // S contains the initial sequence values, 'tr' is the transition array from BM.
63 // K is 0-indexed, S and tr must be same size
64 ll LinearRecurrence(const vector<ll>& S, const vector<ll>& tr, ll k) { //  $O(N^2 \log K)$ 
65     assert(S.size() == tr.size());
66     int n = tr.size();
67     if (n == 0) return 0;
68     if (k < n) return S[k] % MOD;
69
70     vector<ll> pol(n + 1), e(pol);
71     pol[0] = e[1] = 1;
72
73     for (++k; k; k /= 2) {
74         if (k % 2) pol = combine(n, pol, e, tr);
75         e = combine(n, e, e, tr);
76     }
77
78     ll res = 0;
79     for (int i = 0; i < n; i++)
80         res = add(res, mul(pol[i + 1], S[i]));
81     return res;
82 }

```

Product of divisors

```

1 long long productOfDivisors(const vector<pair<long long, long long> > &primeFactors) {
2     // We compute the exponent modulo  $2 * (MOD - 1)$  to safely divide by 2 later
3     long long expMod = 2LL * (MOD - 1);
4     // 1. Calculate the total number of divisors  $d(N)$  modulo  $2 * (MOD - 1)$ 
5     long long d_N = 1;
6     for (const auto &factor: primeFactors) {
7         long long a_i = factor.second;
8         d_N = (d_N * (a_i + 1)) % expMod;
9     }
10    long long product = 1;
11    // 2. Calculate the contribution of each prime factor
12    for (const auto &factor: primeFactors) {
13        long long p_i = factor.first;
14        long long a_i = factor.second;
15        // Exponent for this prime factor:  $(a_i * d_N)$ 
16        long long exponent = (a_i * d_N) % expMod;
17        // Safe division by 2 because  $(a_i * d_N)$  is mathematically guaranteed to be even
18        exponent /= 2;
19        // Multiply the contribution to the final product
20        product = (product * power(p_i, exponent)) % MOD;
21    }
22    return product;
23 }

```